HW6 Readme

Vinisha Bala Dhayanidhi

APPROACH

Part 1 (targets): I wrote the target FOL by hand using the Neo-Davidsonian event semantics. I made sure that each and every verb contributes exists e.(pred(e) & role1(e,x) & ...) with one conjunct per role. So "John eats" has only an eater, while "John eats a sandwich" adds an eaten role.

Part 2 (grammar): The .fcfg follows the textbook and the slides. Every rule has a SEM feature that is built from its children's SEM by function application, e.g. S[SEM=<?subj(?vp)>] -> NP VP.

- NPs are type-raised, <<e,t>,t>. "a" = \Q P.exists x.(Q(x) & P(x)), "every"/"all" = \Q P.all x.(Q(x) -> P(x)), "no" = \Q P.-exists x.(Q(x) & P(x)), names = \P.P(john), and "nobody" = \P.-exists x.(person(x) & P(x)). And giving names the same type as quantified NPs lets one subject rule and one object rule cover both.
- Intransitive verbs: \x.exists e.(eating(e) & eater(e,x)).
- Transitive: \X y.X(\x.exists e.(eating(e) & eater(e,y) & eaten(e,x))), so the object quantifier takes scope over the event.
- A NUM feature handles subject-verb and determiner-noun agreement.
- The boolean operators use NLTK's symbols & | -> -

Part 3 (program): The hw6_semantics.py loads the grammar with FeatureGrammar, parses with FeatureChartParser, and then prints root_semrep(tree).simplify() for the first parse, so that only one reading is written per sentence.

PROBLEMS AND SOLUTIONS

(a) "eats" is both intransitive and transitive, so I gave each verb separate IV and TV entries. This matches the principle that no more roles are postulated than are mentioned.

(b) "or" is NP, nominal, VP and S level, so I added a disjunction rule at each level with semantics matched to the type, e.g. NP:\P.(np1(P) | np2(P)), VP: \x.(vp1(x) | vp2(x)). "and" has the same shape.

(c) "Jack does not eat or drink": my rule VP -> Aux Neg VP[NUM=pl] lets the VP be the disjunction, giving -(eat | drink). The reading (not eat) or drink is blocked by NUM, since "does" is singular and "drink" is plural.

(d) Scope: "nobody eats a bagel" is ambiguous. My grammar is syntax-driven, so I chose one reading: -exists x.(person(x) & exists y.(bagel(y) & exists e.(...))). "no student eats a bagel" is handled the same way.

(e) Capitalization: the program lowercases the first word only if the grammar knows the lowercase form, so "Every" works but "John" stays.

(f) Also, unknown words raise a ValueError in NLTK. The program catches it and writes NO PARSE instead of stopping.

(g) The output variables like z2 and z8 come from alpha-conversion during beta-reduction and are equivalent to my targets.

VERIFICATION
All 15 outputs match my hand-written targets up to variable renaming, and each sentence has exactly one parse, so there is no ambiguitiy.

LIMITATIONS
The lexicon is small and built around the test sentences and there is no tense, no PPs and no bare plurals, and negation only works with do-support. The scope follows the surface syntax only (no Cooper storage), and "and" with NPs is distributive.

WHAT I LEARNED
I saw that type-raising simplifies the grammar, since the real work moves into the lexicon. I noticed that coordination needs a separate semantic rule at each syntactic level and features like NUM can also remove wrong readings, not just ungrammatical strings.